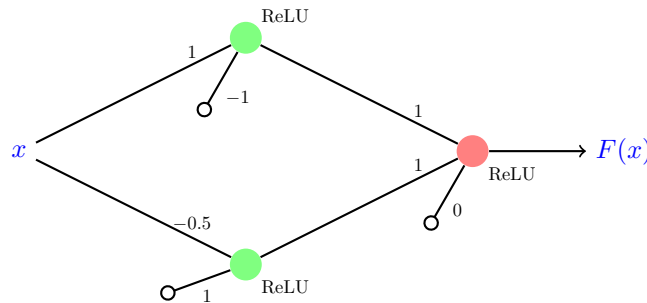


TD n°1 : Réseaux de neurones multicouches (*tensorflow* avec *keras*)

Ce TD introduit le fonctionnement d'un perceptron multi-couches, à partir du module *keras*. A titre d'exemple, on crée le réseau de neurones suivant :



Description du module *keras* de *tensorflow*

Outre l'importation de *numpy*, il est nécessaire d'importer le sous-module *keras* du module *tensorflow*, accompagné de quelques outils spécifiques :

```
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense
```

On peut alors définir l'architecture de ce réseau simple, en le décrivant couche par couche :

```
# Architecture du reseau
modele = Sequential()

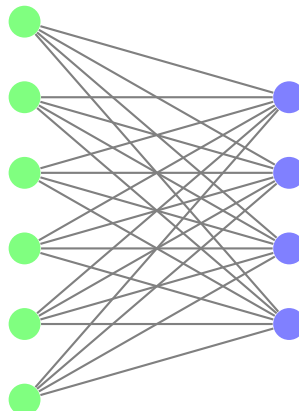
# Definir une entree de dimension 1
modele.add(Input(shape=(1,)))

# Couches de neurones
modele.add(Dense(2, activation='relu'))
modele.add(Dense(1, activation='relu'))
```

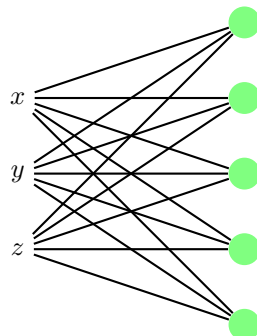
Explications et commentaires

- Le réseau s'appelle ici `modele`, il est du type `Sequential`, c'est-à-dire décrit par une suite de couches les unes à la suite des autres.
- Chaque couche est ajoutée à la précédente par `modele.add()`. L'ordre dans lequel se fait chaque ajout est donc pertinent. Chaque couche est ajoutée par une commande de profil général :

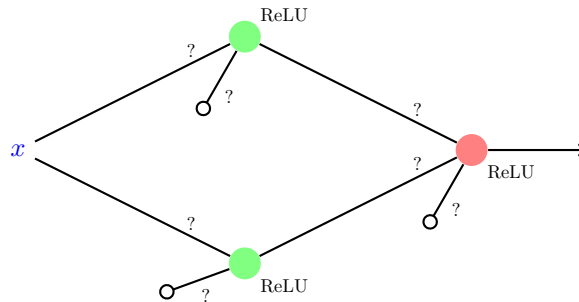
```
modele.add(Dense(nb_neurones, activation=ma_fonction))
```
- Une couche de type `Dense` signifie que chaque neurone de la nouvelle couche est connecté à toutes les sorties des neurones de la couche précédente :



- Pour chaque couche, il est nécessaire de préciser le nombre de neurones qu'elle contient : s'il y a n neurones, alors la couche renvoie n valeurs en sortie ; un neurone renvoie la même valeur de sortie à tous les neurones de la couche suivante.
- On précise le nombre de valeurs en entrée de la première couche par l'option `input_dim=...` ; dans le code donné ici, on a une entrée d'une seule variable. La figure ci-dessous est un exemple d'une entrée en dimension 3 :



- Pour les autres couches, le nombre d'entrées est égal au nombre de sorties de la couche précédente ; il n'est donc pas la peine de le préciser.
- Pour chaque couche, il faut également préciser une fonction d'activation (c'est la même pour tous les neurones d'une même couche). Plusieurs fonctions d'activation sont prédéfinies :
`'relu'` (ReLU), `'sigmoid'` (σ), `'linear'` (identité)
- L'exemple ne possède qu'une entrée et comme la dernière couche n'est constituée que d'un neurone, il n'y a qu'une valeur de sortie. Ainsi, ce réseau définit une fonction $x \mapsto F(x)$.
- A ce stade, il s'agit d'un *modèle* de réseau puisqu'aucun poids n'a été encore fixé...



Pour vérifier que tout est correct jusque là, il est possible d'exécuter la commande `modele.summary()` qui affiche un résumé des couches et du nombre de poids à définir.

Les poids

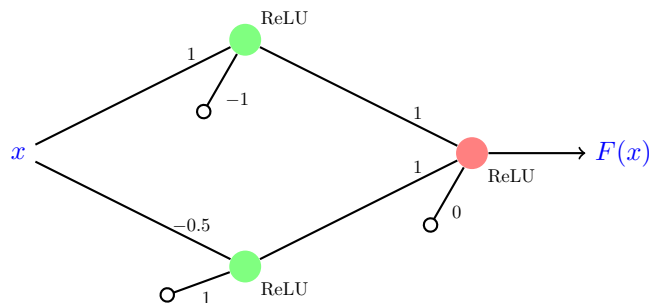
Lors de la définition d'un réseau et de la structure de ses couches, des poids aléatoires sont attribués à chaque neurone. La démarche habituelle est ensuite d'entraîner le réseau, automatiquement, afin qu'il trouve de « bons » poids. Mais pour l'instant, nous continuons de fixer les poids de chaque neurone à la main. La commande pour fixer les poids est `set_weights()`.

Voici la définition des poids de la première couche, numérotée 0 :

```
# Couche 0
coeff = np.array([[1., -0.5]])
biais = np.array([-1, 1])
poids = [coeff, biais]
modele.layers[0].set_weights(poids)
```

Pour les poids de la couche numéro 1 :

```
# Couche 1
coeff = np.array([[1.0], [1.0]])
biais = np.array([0])
poids = [coeff, biais]
modele.layers[1].set_weights(poids)
```



Quelques précisions concernant la commande `set_weights()`, dont l'utilisation n'est pas aisée d'emblée... :

- Les poids sont définis pour tous les éléments d'une couche, par une commande `set_weights(poids)`.
- Les poids sont donnés sous la forme d'une liste : `poids = [coeff, biais]`.
- Les biais sont donnés sous la forme d'un vecteur de biais (un pour chaque neurone).
- Les coefficients sont donnés sous la forme d'un tableau à deux dimensions. Ils sont définis par entrée. La structure n'en est pas naturelle.

Pour vérifier que les poids d'une couche sont corrects, on utilise la commande `get_weights()`, par exemple pour la première couche : `modele.layers[0].get_weights()`

Cette instruction renvoie les poids sous la forme d'une liste [coefficients,biais] du type :

```
[ [[ 1. -0.5]], [[-1. 1.] ]
```

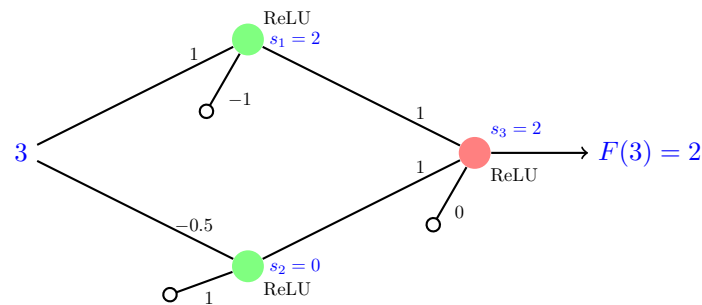
Remarque. Cette commande est aussi très pratique (avant même de fixer les poids) pour savoir quelle est la forme que doivent prendre les poids afin d'utiliser `set\_weights()`.

Evaluation

On utilise le réseau avec `predict()`. Notre réseau définit une fonction $x \mapsto F(x)$. L'entrée correspond à un réel et la sortie également :

```
entree = np.array([[3.0]])
sortie = modele.predict(entree)
```

Ici `sortie` vaut `[[2.0]]` et donc $F(3) = 2$. Ce que l'on peut vérifier à la main en calculant les sorties de chaque neurone.



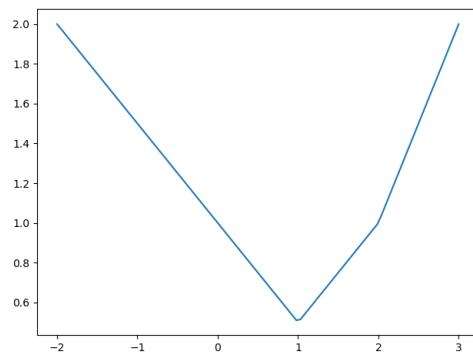
Visualisation

Afin de tracer le graphe de la fonction F , on peut calculer d'autres valeurs :

```
import matplotlib.pyplot as plt
liste_x = np.linspace(-2, 3, num=100)
entree = np.array([[x] for x in liste_x])

sortie = modele.predict(entree)

liste_y = np.array([y[0] for y in sortie])
plt.plot(liste_x, liste_y)
plt.show()
```



Exercice Réaliser et tester le réseau de neurones suivant (rendu à effectuer sous forme d'un notebook jupyter ou d'un .pdf) :

